The Building Blocks of LLMs

A Mathematical History

From Archimedes to Attention:
2,000+ Years of Mathematics Inside Your AI

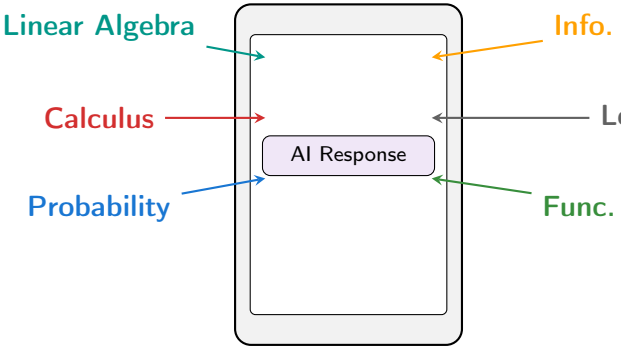
Prof. J. Osterrieder 2026

What's Inside Your AI?

When you type a question to ChatGPT or Claude...

The machine that answers is built entirely from mathematics.

- Not magic
- Not "intelligence"
- **Specific math**, invented by **specific people**, at **specific times**

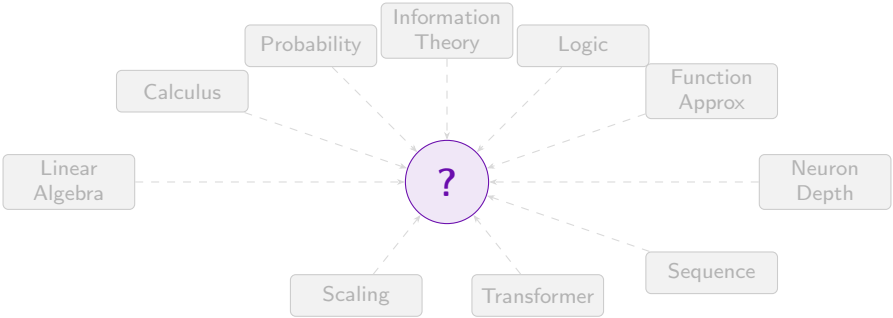


Key Insight

You already know some of this math

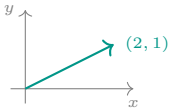
The Roadmap — Ten Streams, One Machine

Building Blocks Found: 0 / 10



Ten branches of mathematics. 2,000+ years. All converging in one architecture.

Vectors, Matrices, and Words



2D

$$\begin{pmatrix} 0.23 \\ -0.87 \\ 0.44 \\ \vdots \\ 0.12 \end{pmatrix}$$

768 numbers

A **vector** = list of numbers. In an LLM, each word becomes a vector of 768+ numbers.

A **matrix** = grid of numbers. Each neural network layer = matrix multiplication.

$$\begin{pmatrix} 2 & 0 \\ 3 & 1 \end{pmatrix}$$ Cayley, 1858

$\longrightarrow$ **175 billion entries.**
Same math. Just bigger.

The Inventors

Grassmann (1844)
Schoolteacher. First general vector spaces. Nobody read his book.

Cayley (1858)
Lawyer-turned-mathematician. Matrices as algebra.

Sylvester (1850)
Coined "matrix" from Latin for "womb."

Gauss (1809)
Gaussian elimination — solving linear systems.

Inside the LLM

96 layers $\times$ matrix multiply = how ChatGPT generates every word.

The Dot Product — How AI Measures Similarity

Dot Product (Gibbs, 1881)

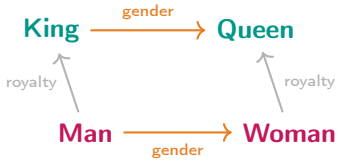
$$\vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

Similar → large positive
Unrelated → near zero
Opposite → large negative

Interactive Moment

$$\vec{\text{King}} - \vec{\text{Man}} + \vec{\text{Woman}} = ?$$

What word do you get?



Mikolov et al., 2013: Word2Vec

The Embedding Pipeline

`"Hello world"`
↓ tokenize
`[Hello] [world]`
↓ embed

$$\begin{pmatrix} 0.2 \\ -0.8 \\ \vdots \end{pmatrix} \begin{pmatrix} 0.9 \\ 0.1 \\ \vdots \end{pmatrix}$$

Linear Algebra — The Skeleton Assembled

Building Blocks Found: 1 / 10

Section Summary

Inventor	Contribution
Grassmann (1844)	General vector spaces
Sylvester (1850)	Named "matrix"
Cayley (1858)	Matrix algebra
Gibbs (1881)	Dot product
Gauss (1809)	Linear systems
Mikolov (2013)	Word embeddings

Key Insight

Every layer is a matrix multiplication.

Every attention score is a dot product.

The math of the 1840s is the engine of 2026.

One node lit. Nine to go.

A skeleton doesn't move.
For that, we need ***calculus*** *— the mathematics of change.*

The Derivative and Gradient Descent

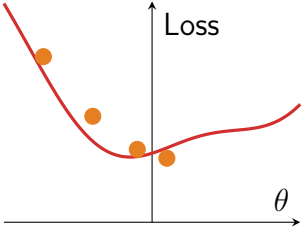
The Calculus Chain

Archimedes (c. 250 BCE) – method of exhaustion
Fermat (1630s) – tangent lines
Newton & Leibniz (1660s–80s) – the derivative
Cauchy (1847) – steepest descent

Gradient Descent Recipe

1. Compute the slope (gradient)
2. Take a step downhill
3. Repeat

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot \nabla L(\theta)$$



Rolling downhill on the loss landscape.
Cauchy, 1847.

Backpropagation — The Chain Rule Saves AI

The Chain Rule

$$\frac{df}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

Derivative of a composition
= product of individual derivatives

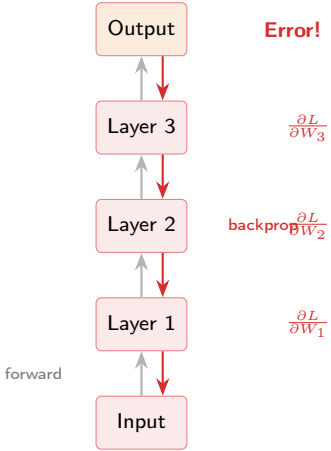
Leibniz notation, 1684

Timeline

Leibniz (1684) – the notation

Linnainmaa (1970) – automatic differentiation

Rumelhart, Hinton, Williams (1986) – backprop in neural nets



Calculus — The Learning Engine Assembled

Building Blocks Found: 2 / 10

Section Summary

Inventor	Contribution
Archimedes (c. 250 BCE)	Proto-calculus
Newton/Leibniz (1680s)	The derivative
Cauchy (1847)	Gradient descent
Linnainmaa (1970)	Auto-differentiation
Hinton et al. (1986)	Backprop in NNs

Key Insight

Without calculus, you can *build* a neural network but you cannot *train* it.

Two nodes lit.

The network has a skeleton and can learn.
*But learn **what**?*

*It learns to predict **probability distributions** over words.*
For that, we need probability theory.

From Gambling to Prediction

Pascal
Paris

1654

Fermat
Toulouse

Bayes (1763)

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

Update beliefs from evidence.
LLMs learn: $P(\text{next word} \mid \text{context})$

Shannon (1948) — Language as Statistics

0th order: XFOML RXKHRJFFJUJ

1st order: OCRO HLI RGWR

2nd order: ON IE ANTSOUTINYS

Word bigram: THE HEAD AND IN FRONTAL

More context = more readable.
This IS what every LLM does.

Inside the LLM

Every word your AI generates is chosen by probability — a distribution over 50,000+ possible next tokens.

Softmax — From Thermodynamics to Token Prediction

Boltzmann (1870s)

Energy distribution of gas molecules:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

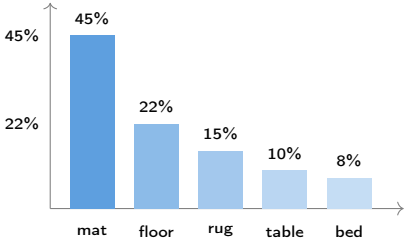
Same formula. Now picks words.

Raw scores (logits) → e^{z_i} → Normalize $\sum = 1$

Interactive Moment

"The cat sat on the _____"

What word comes next?



Cross-Entropy — Measuring How Wrong the AI Is

Building Blocks Found: 4 / 10

Shannon's Loss Function (1948)

Cross-entropy loss $= -\log\big(p(\text{correct word})\big)$

	Confidence	Loss
"mat" at 30%	0.30	1.20
"mat" at 80%	0.80	0.22

Training = minimizing cross-entropy = making the model more confident about the RIGHT word.

Key Insight

Shannon's 1948 paper contributes to **two** building blocks: language modeling (probability) AND the loss function (information theory).

Four nodes lit.

Prediction math: probability
Scoring math: information theory
Learning math: calculus
Structure math: linear algebra

*But we need something to **run it on**.*

Logic & Computation — The Substrate

The Logic Chain

Boole (1854) — Logic as algebra.
True/false = 1/0. Son of a shoemaker.

Turing (1936) — Universal machine.
Can compute anything computable.

Von Neumann (1945) — Stored program.
The architecture of every computer.

NVIDIA CUDA (2007) — GPUs.
Billions of parallel operations.

AND	OR	NOT
$1 \wedge 0 = 0$	$1 \vee 0 = 1$	$\neg 1 = 0$

The Paradox

Gödel and Turing proved that no formal system can capture all truth.

Yet neural networks — running on the hardware that logic built — learned to do things formal logic cannot:

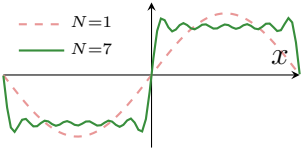
translate, write poetry, reason by analogy.

The machines that logic built transcended the limitations that logic discovered.

"Boole was a self-taught son of a shoemaker who reduced logic to algebra. Turing imagined a machine that could

Function Approximation — The Guarantee

Building Blocks Found: 6 / 10



Any function = sum of waves. *Fourier, 1807.*

The Chain of Guarantees

Fourier (1807)
Any function as a sum of sines.

Weierstrass (1885)
Polynomials can approximate anything.

Cybenko (1989)
Neural networks can approximate *any* continuous function.

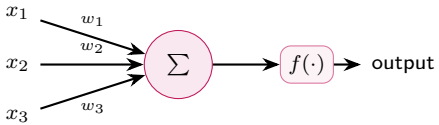
A neural network **can** learn any continuous function. This is not an analogy. It is a *mathematical theorem*.

Six nodes lit. We know they CAN learn anything.

The Neuron — From Biology to Mathematics

McCulloch & Pitts (1943)

A neurophysiologist and a **teenage runaway** (self-taught logician, no degree) created the mathematical neuron.



Rosenblatt's Perceptron (1958)

A room-sized machine with 400 photocells that could **learn**.

The New York Times: "a machine that could walk, talk, see, write. . ."

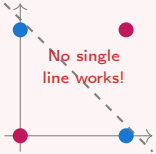
The hype was enormous.

And then came the crash.

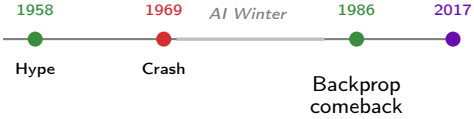
Repeat until correct

The Crash and Comeback

The XOR Problem (1969)



Minsky & Papert proved:
one layer cannot learn XOR.



The Solution: More Layers

Minsky & Papert's math was correct.
Their conclusion was wrong.

The solution: **add more layers.**

Rumelhart, Hinton & Williams (1986):
backpropagation can train multi-layer networks.

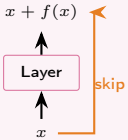
The chain rule saved an entire research program.

Depth — Activation, Residuals, Normalization

Activation

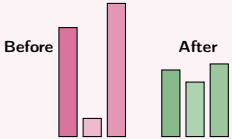
Without this,
100 layers = 1 layer.

Residuals (He, 2015)



Signal doesn't fade.

LayerNorm (Ba, 2016)



Numbers don't explode.

"Add & Norm" in the transformer = He's residual + Ba's normalization.
You will see them on slide 23.

The Neuron & Depth — Seven Nodes Lit

Building Blocks Found: 7 / 10

Section Summary

Inventor	Contribution
McCulloch & Pitts (1943)	Mathematical neuron
Rosenblatt (1958)	Perceptron (learning)
Minsky & Papert (1969)	XOR impossibility
Hinton et al. (1986)	Backprop revival
Nair & Hinton (2010)	ReLU activation
He et al. (2015)	Residual connections
Ba et al. (2016)	Layer normalization

Key Insight

Billions of neurons, stacked 96+ layers deep, with activation, residuals, and normalization.

But language is **sequential**.
"Dog bites man" $\neq$ "Man bites dog."

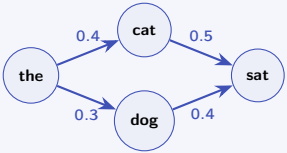
Seven nodes lit.

We can build neurons, stack them deep, and train them.

*But how do you process a **sequence**?*

From Markov Chains to Attention

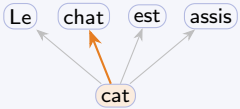
Markov (1906)



Analyzed Pushkin's poetry.
Next word depends only on current word.

Attention (Bahdanau, 2014)

Instead of compressing everything into one vector, **look back at the whole input.**



The model **chooses** what to attend to.

The Problem: Memory Fades

RNNs: memory fades after ∼50 words.
The beginning of a paragraph is forgotten.

Without this 2014 paper, there would be no GPT, no Claude.

Attention — The Bridge to the Transformer

Building Blocks Found: 8 / 10

Eight Nodes Lit

Two thousand years of mathematics.

Linear algebra, calculus, probability, information theory, logic, function approximation, the neuron, sequences.

All of these streams are about to converge in a single architecture.

Two nodes remain.

Published by eight researchers at Google in 2017.

Most in their 20s and 30s.

“Attention Is All You Need” (2017)

arXiv:1706.03762

Attention Is All You Need

Ashish Vaswani Noam Shazeer Niki Parmar
Jakob Uszkoreit Llion Jones Aidan N. Gomez
Łukasz Kaiser Illia Polosukhin

Google Brain / Google Research 2017

Eight researchers. One paper.
The architecture that powers every major LLM.

Most in their 20s and 30s. Several founded major AI companies.

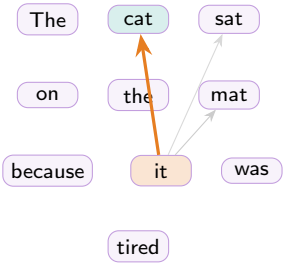
Self-Attention — The Core Mechanism

Each Token Generates Three Vectors

Q (Query): What am I looking for?
K (Key): What do I contain?
V (Value): What information do I carry?

Self-Attention Steps

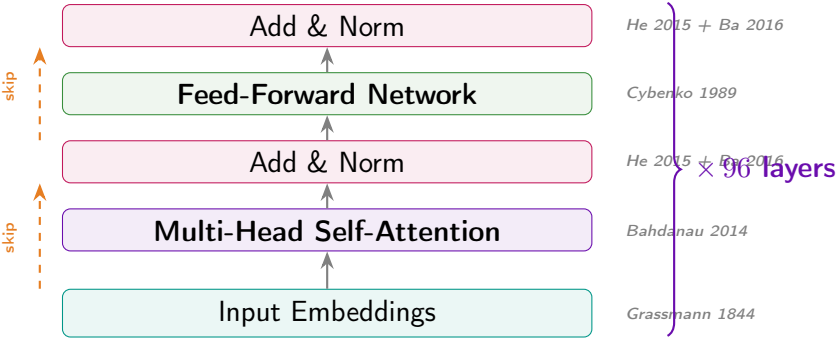
1. Dot product $Q \cdot K^T$ (Gibbs, slide 5)
2. Scale by $1/\sqrt{d}$
3. Softmax (Boltzmann, slide 11)
4. Multiply by V (Cayley, slide 4)



"it" attends to "cat"

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

One Transformer Layer — The Full Stack



Every component was taught in an earlier slide.

The Full Annotated Transformer

Output Softmax	Boltzmann 1870s
Add & Norm	
FFN + GELU	Cybenko 1989
Add & Norm	He 2015, Ba 2016
Self-Attention ($Q \cdot K^T / V$)	Gibbs + Boltzmann
Positional Encoding	Fourier 1807
Input Embedding	Grassmann 1844

Training: SGD (Cauchy 1847) + Backprop (Linnainmaa 1970)

Loss: Cross-entropy (Shannon 1948)

Interactive Moment

Count the building blocks.

How many of the 10 sections can you find in this diagram?

Each component was invented by someone you just met.

2,000 years of mathematics, all running at once.

1807 · 1844 · 1847 · 1858 · 1870s · 1881
1943 · 1948 · 1970 · 1986 · 1989
2013 · 2014 · 2015 · 2016 · 2017

Positional Encoding — Fourier Inside the Transformer

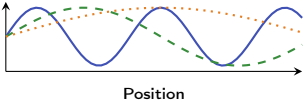
Same Math, 210 Years Apart

Fourier (1807): Decompose heat into waves.

Vaswani (2017): Decompose word position into waves.

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$



Different frequencies = different position scales.

Building Blocks Found: 9 / 10

Nine nodes lit. One to go.

Key Insight

Your AI knows that "dog bites man" and "man bites dog" are different because of positional encoding.

Fourier's sine waves, repurposed.

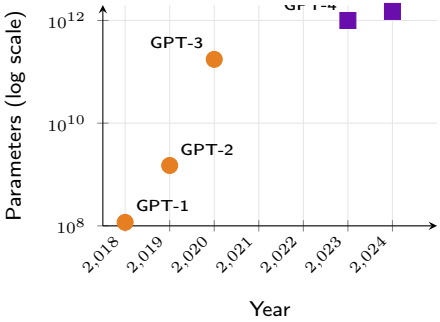
The architecture is ready.
Now: scale it up.

Scaling Laws — Same Architecture, Just Bigger

The Scaling Explosion

Model	Year	Parameters
GPT-1	2018	117M
GPT-2	2019	1.5B
GPT-3	2020	175B
GPT-4	2023	undisclosed
Claude 3.5	2024	undisclosed

Same 2017 blueprint. 1,500× larger.



Kaplan et al. (2020): performance scales as a smooth power law with parameters, data, and compute.

Return to the Diagram — Now You Understand

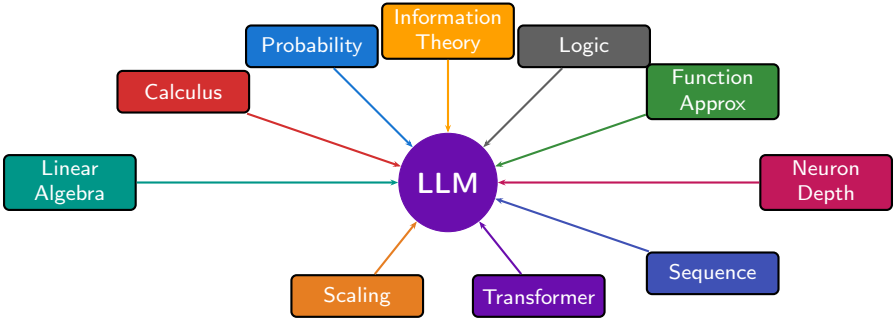
The Complete Inventory

Component	Inventor(s)	Date
Embedding	Grassmann, Mikolov	1844, 2013
Positional encoding	Fourier, Vaswani	1807, 2017
Self-attention (dot product)	Gibbs, Bahdanau	1881, 2014
Softmax	Boltzmann	1870s
Matrix multiplication	Cayley	1858
Add & Norm	He, Ba	2015, 2016
Feed-forward + GELU	Cybenko, Hendrycks	1989, 2016
Output softmax	Boltzmann/Luce	1870s
Gradient descent	Cauchy	1847
Backpropagation	Linnainmaa, Hinton	1970, 1986
Cross-entropy loss	Shannon	1948

At the start, this was overwhelming. Now you can point to each piece and say **who** invented it, **when**, and **why**.

Full Convergence — All Ten Streams

Building Blocks Found: 10 / 10



Ten streams of human invention, flowing into one machine.

The Story Is Not Over

Open Questions

Why do transformers work so well?
No one fully knows.

What are the limits of scaling?
Are we near the ceiling or far from it?

What comes after transformers?
State space models? Hybrid architectures?

What new math will be needed?
The tools may not exist yet.

Perspective

The math is 2,000 years old.

The architecture is only **8 years old**.

There are open questions everywhere.

Every breakthrough we covered today started with someone asking a question nobody else was asking.

The next breakthrough will start the same way.

1943

McCulloch & Pitts draw
a single neuron on paper.

2026

Trillion-parameter models
reason, write, and create.

The math your generation will create is not written yet.

2,000+ Years of Mathematics. One Machine. **Your Turn.**